

Idiomatic Go & Project Architecture Cheatsheet

Lessons: [24 — Idiomatic Go](#) · [25 — Project Layout & Clean Architecture](#) · [26 — Capstone](#)

Examples: —

Covers: naming, style rules, error conventions, `vet` /lint, layering, dependency injection, the review checklist

Legend: `[*]` = tool or convention the lessons have not covered yet

NAMING

MixedCaps / mixedCaps	never underscores, never SCREAMING_CASE
userID, not userId	initialisms stay uppercase: ID, URL, HTTP, API
short names, short scopes	i, r, w, buf, err in a 5-line function
long names, long lives	exported identifiers explain themselves
no stutter	user.Service, not user.UserService
package name is a prefix	store.New reads as "store.New" at the call site
New / NewThing	constructors: New if the package has one type
Interface names end in -er	Reader, Writer, Handler, Formatter
one-method interface	name it after the method: Read -> Reader
no Get prefix	u.Name(), not u.GetName(); Set stays Set
err	the error variable, always
ctx	the context variable, always, and always first

STYLE

gofmt decides	tabs, alignment, brace placement – no discussion
goimports	<code>[*]</code> gofmt plus import grouping and pruning
early return	the happy path stays at the left margin
if err != nil { return ... }	handle it immediately, don't accumulate
no else after return	flatten it
one concept per function	if you need "and" to name it, split it
accept interfaces, return structs	flexible in, concrete out
zero value should work	var b bytes.Buffer, var mu sync.Mutex – no New needed
make the zero value useful	or force a constructor, but don't do half of each
comments explain WHY	the code already says what
doc comments start with the name	// User represents an account holder.
package comment in doc.go	<code>[*]</code> for anything non-trivial
no naked returns in long funcs	readable in 3 lines, opaque in 30

ERRORS (the conventions)

return errors, don't panic	panic is for programmer bugs only
error is the LAST result	always
wrap with context	<code>fmt.Errorf("load user %d: %w", id, err)</code>
lowercase, no trailing punctuation	they get concatenated
sentinel for expected cases	<code>var ErrNotFound = errors.New("not found")</code>
typed error when you need data	status codes, field names, IDs
errors.Is / errors.As	never == once anything wraps
handle OR return, never both	the caller logs it once, at the top
<code>_ = f()</code>	explicit ignore, with a comment saying why
(the error message read top to bottom should tell the whole story)	

THE TOOLS

gofmt -l ./...	list unformatted files (CI gate)
go vet ./...	printf mismatches, copied locks, unreachable code
staticcheck	[*] the deep static analyzer
golangci-lint run	[*] runs many linters at once; config in .golangci.yml
errcheck	[*] unchecked error returns
ineffassign	[*] assignments that are never read
govet / staticcheck	[*] the core two
revive	[*] style rules
govulncheck ./...	[*] known CVEs in your dependency graph
go test -race	the one that finds real bugs
go doc ./...	read your own package the way callers will

PROJECT LAYOUT

cmd/api/main.go	one directory per binary; main is TINY
internal/	compiler-enforced privacy – put almost everything here
internal/domain/	entities and business rules, no imports of infra
internal/service/	use cases; orchestrates domain + ports
internal/repository/	the DB implementations
internal/handler/	HTTP: decode, call service, encode
internal/config/	Load() and the Config struct
migrations/	SQL, numbered
testdata/	fixtures, ignored by the toolchain
pkg/	[*] only if outside code genuinely imports it

(start flat with 3 files; add a directory when naming gets hard)

LAYERING

handler -> service -> repository -> database	the dependency direction
domain imports NOTHING	not net/http, not database/sql, not your framework
the service owns the use case	one method per user-visible operation
the handler owns HTTP	status codes, JSON, headers – nothing else
the repository owns SQL	it never leaks *sql.Rows upward
interfaces live in the CONSUMER	the service declares what it needs
DTOs at the edges	request/response structs are the API contract
map DTO <-> domain	explicitly, in the handler

(a change to the database should never force a change to the domain)

DEPENDENCY INJECTION

main is the composition root everything is constructed there, in order

```
cfg := config.Load()
db := db.Connect(cfg)
repo := repository.NewUserRepo(db)
svc := service.NewUserService(repo)
h := handler.NewUserHandler(svc)
```

constructor injection	pass dependencies in, store them in the struct
no globals	var db *sql.DB at package level is the anti-pattern
no service locator	a container you ask for things is a global in disguise
interfaces for what varies	the DB, the clock, the mailer – not everything
manual DI is enough	wire/fx solve a problem you probably don't have

(if a test needs a real database, the wiring is wrong, not the test)

THE REVIEW CHECKLIST

- [] gofmt clean, go vet clean, tests pass with -race
- [] every error is handled, wrapped with context, or explicitly ignored
- [] ctx is the first parameter and is actually passed through
- [] no goroutine without a guaranteed exit
- [] no data shared between goroutines without a mutex or a channel
- [] the zero value works, or a constructor is required
- [] exported identifiers have doc comments
- [] interfaces are small and defined by the consumer
- [] no globals, no init() side effects
- [] the domain package imports no infrastructure
- [] table-driven tests for the branches that matter
- [] no secrets in logs, no SQL built with Sprintf
- [] timeouts on every server and every outbound client

PROVERBS WORTH REMEMBERING

Clear is better than clever.

Don't communicate by sharing memory; share memory by communicating.

The bigger the interface, the weaker the abstraction.

Make the zero value useful.

A little copying is better than a little dependency.

errors are values.

Don't just check errors, handle them gracefully.

Design the architecture, name the components, document the details.

Documentation is for users.

gofmt's style is nobody's favorite, yet gofmt is everybody's favorite.

TRAPS & MEMORIZE

package util / common / helpers	a bag with no design in it
interfaces defined next to the impl	the wrong side; the consumer decides
one giant interface	impossible to fake; split by use case
premature abstraction	write it concrete twice, then generalize
globals for the DB/logger	untestable, and concurrent tests collide
init() with side effects	hidden ordering, impossible to disable
fat main()	wiring is fine; logic is not
domain importing net/http	the layering has already failed
returning the DB model as JSON	every column change is an API change
panic for validation	a 400 is not an exceptional condition
ignoring go vet	it only reports things that are actually wrong